as well if the queue has less than `n` elements: since at line #3 we added a tuple with distance equal to infinity, that tuple will be at the top of the heap until we add `n` points, and so `nnnDist` will be set to infinity here, as long as we haven't yet added at least `n` points to the queue.

#16 Using one of the helper functions defined in listing 9.2, we compute the distance between the split line passing through current node and the target point. If this distance is closer than the distance to current `n`-th nearest neighbor, as it's stored in the queue, then the furthest branch might contain closer points.

#17 Traverse furthest branch, and update current values for NNs and their distance.

#18 Return the priority queue with the points found so far.

What's the running time for nearest neighbor search? Let's start from the bad news: in the worst-case scenario, even for a balanced tree, we might have to traverse the whole tree before finding a point's nearest neighbor(s).

Figure 9.23 shows a couple of examples of such a degenerate case: while the second example is artificially constructed as a literal edge case, with all the points lying on a circle, the one in figure 9.23A, showing the same tree we used in our examples above, demonstrates how even on random, balanced, trees it is possible to find counter-examples where the method behaves poorly by just carefully choosing the target point for the search.